0518. Coin Change II

| Category: 2-D Dynamic Programming | LeetCode: https://leetcode.com/problem

Problem

- Count the combinations (order does not matter) of coins that sum to amount. Unlimited supply.

Intuition

- Process coins one at a time in the outer loop: ways[a] += ways[a - coin]. Since each coin is introduced once, combinations are counted in a fixed coin order, so permutations are not double counted.

Brute Force

- Backtracking with a non-decreasing coin index: exponential.

Optimal Approach

- Ways[0] = 1.
- For coin c: for a = c..amount: ways[a] += ways[a - c].

Worked Example

```
amount 5, coins [1,2,5].

after 1: ways = 1 1 1 1 1 1
after 2: ways = 1 1 2 2 3 3
after 5: ways = 1 1 2 2 3 4   -> 4
```

Java Solution

```java
public class CoinChangeII {
    public int change(int amount, int[] coins) {
        int[] ways = new int[amount + 1];
        ways[0] = 1;
        for (int c : coins)
            for (int a = c; a <= amount; a++) ways[a] += ways[a - c];
        return ways[amount];
    }
}
```

Complexity

- Time: O(amount coins). Space: O(amount).

Pitfalls

- Swapping the loops (amount outer, coins inner) counts permutations instead (that is Combination Sum IV).